

CH4 – Clip 12.

CI/CD 실습



01

환경 구축

02

Jenkins Hello World

03

Jenkins 예시

1. 환경 구축

환경 구축

Jenkins 환경 구축

- Docker를 활용해 jenkins환경을 구축하고 실습을 진행
- 1. docker compose 파일 작성 (docker-compose.yml)
- 2. 커맨드 실행 : docker-compose up
- <http://localhost:8080> 접속

Jenkins 비밀번호 확인

- 컨테이너 ID 확인 -> 커맨드 실행 -> docker ps
- 커맨드 실행 -> docker logs [컨테이너 ID]
- 비밀번호 확인 -> "Please use the following password to proceed to installation" 다음에 문자열이 초기 관리자 비밀번호

docker-compose.yml

```
version: '3.8'
services:
  jenkins:
    image: jenkins/jenkins:its
    ports:
      - "8080:8080"
      - "50000:50000"
    volumes:
      - jenkins_home:/var/jenkins_home
    environment:
      - JENKINS_OPTS=--httpPort=8080
    restart: unless-stopped

volumes:
  jenkins_home:
```

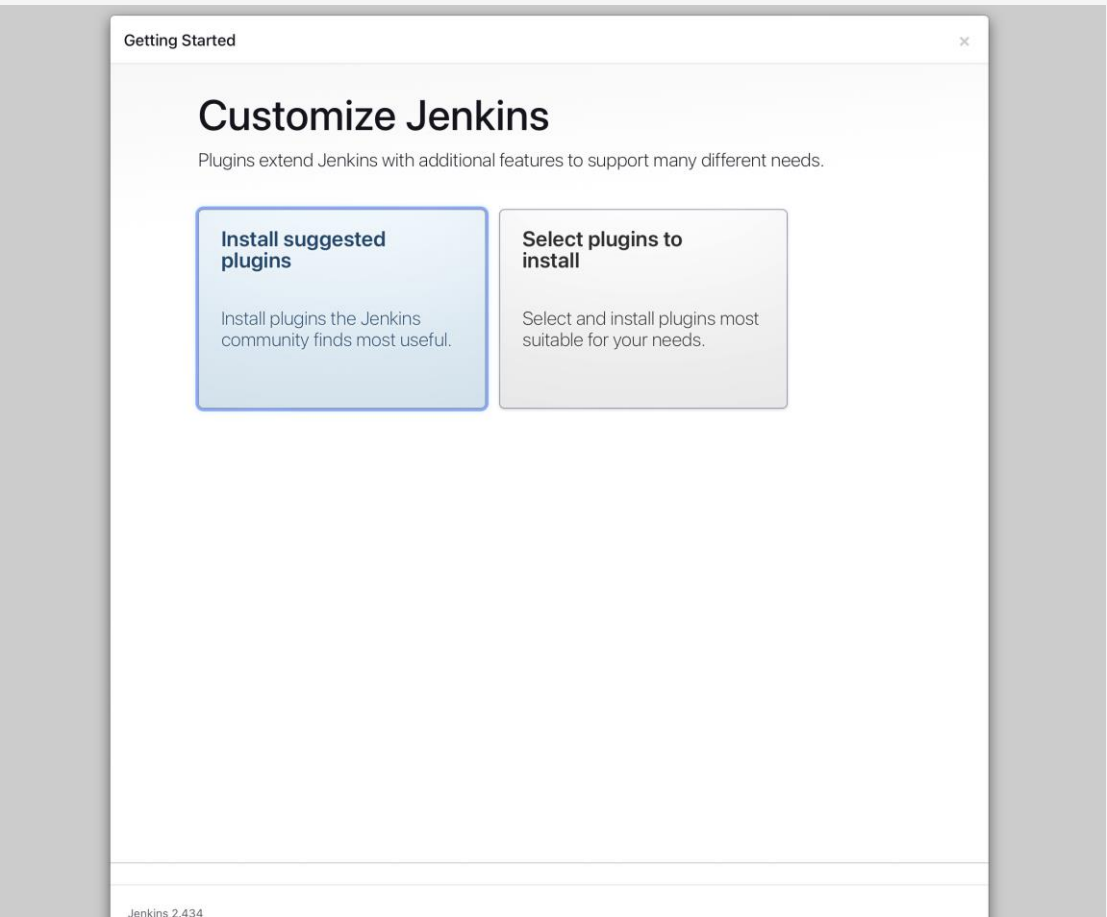
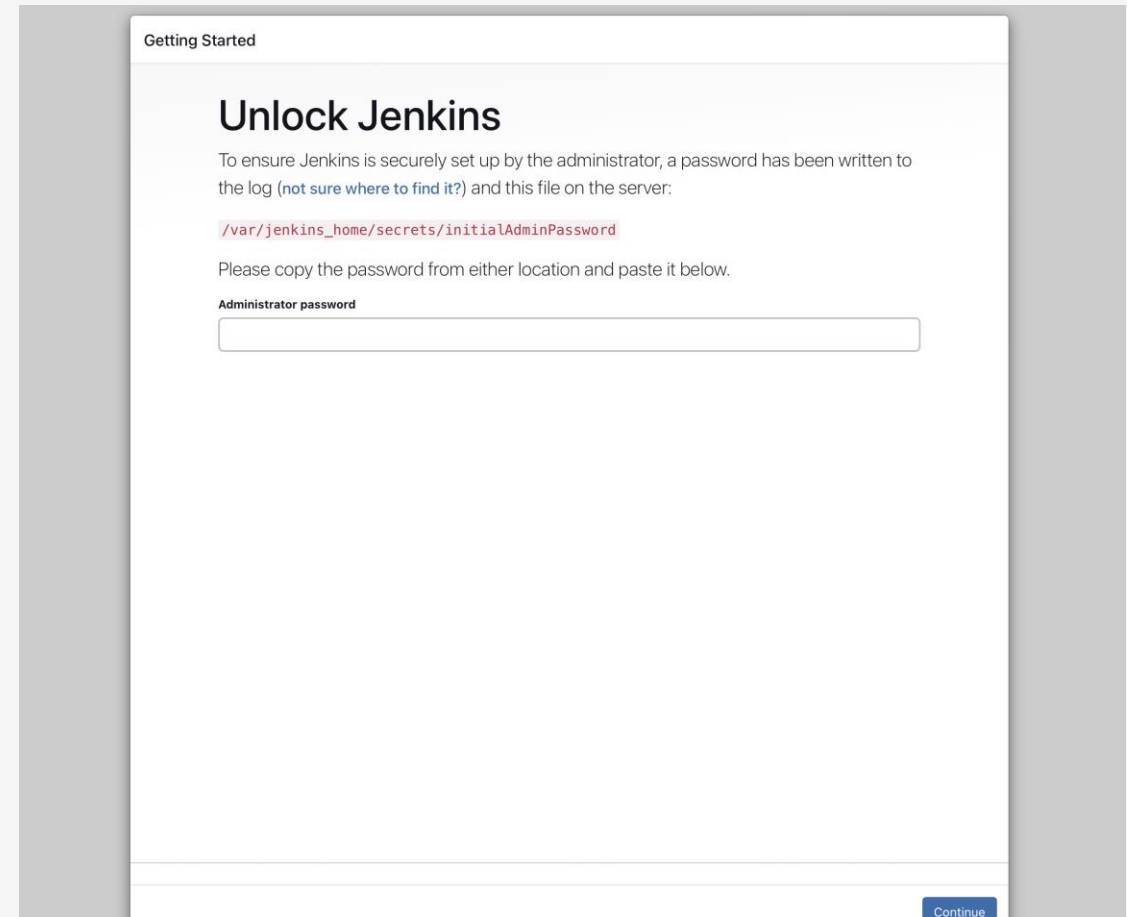
```
*****
*****
*****
```

Jenkins initial setup is required. An admin user has been created and a password generated.
Please use the following password to proceed to installation:

0105c5f98b4d4291aa5e799dda9f64e9

This may also be found at: /var/jenkins_home/secrets/initialAdminPassword

```
*****
*****
*****
```



Getting Started

Getting Started

 Folders	 OWASP Markup Formatter	 Build Timeout	 Credentials Binding	** Icons API
 Timestamp	 Workspace Cleanup	 Ant	 Gradle	
 Pipeline	 GitHub Branch Source	 Pipeline: GitHub Groovy Libraries	 Pipeline: Stage View	
 Git	 SSH Build Agents	 Matrix Authorization Strategy	 PAM Authentication	
 LDAP	 Email Extension	 Mailer		
				** - required dependency

Jenkins 2.434

Getting Started

Getting Started

✓  Folders	✓  OWASP Markup Formatter	✓  Build Timeout	✓  Credentials Binding	** Icons API
 Timestamp	 Workspace Cleanup	 Ant	 Gradle	OWASP Markup Formatter
 Pipeline	 GitHub Branch Source	 Pipeline: GitHub Groovy Libraries	 Pipeline: Stage View	** Structs ** bouncycastle API ** Instance Identity ** JavaBeans Activation Framework (JAF) API ** JavaMail API ** Pipeline: Step API ** Token Macro
 Git	 SSH Build Agents	 Matrix Authorization Strategy	 PAM Authentication	Build Timeout
 LDAP	 Email Extension	 Mailer		** Credentials ** Plain Credentials ** Trilead API ** SSH Credentials Credentials Binding ** SCM API
				** - required dependency

Jenkins 2.434

Getting Started

Create First Admin User

계정명

암호

암호 확인

이름

이메일 주소

Jenkins 2.426.1

Skip and continue as admin

Save and Continue

Getting Started

Instance Configuration

Jenkins URL:

http://localhost:8080/

The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.

The proposed default value shown is **not saved yet** and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.

Jenkins 2.426.1

Not now

Save and Finish

**Jenkins**

Q 검색 (⌘+K) ?

 1  jenkins ▾  로그아웃

Dashboard >

+ 새로운 Item

 사람

 빌드 기록

 Jenkins 관리

 My Views

빌드 대기 목록 ▾

빌드 대기 항목이 없습니다.

빌드 실행 상태 ▾

1 대기 중

2 대기 중

Jenkins에 오신 것을 환영합니다.

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

Start building your software project

Create a job +

Set up a distributed build

Set up an agent 

Configure a cloud 

Learn more about distributed builds ?

 상세 내용 입력

REST API Jenkins 2.426.1

2. Jenkins hello world

REST API Jenkins 2.426.1

 Jenkins

 ?

 1

 jenkins

 로그아웃

Dashboard > All >

Enter an item name

» Required field

1. Item name 입력

 **Freestyle project**

이것은 Jenkins의 주요 기능입니다. Jenkins은 어느 빌드 시스템과 어떤 SCM(형상관리)으로 묶인 당신의 프로젝트를 빌드할 것이고, 소프트웨어 빌드보다 다른 어떤 것에 자주 사용될 수 있습니다.

 **Pipeline**

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

 **Multi-configuration project**

다양한 환경에서의 테스트, 플레폼 특성 빌드, 기타 등등 처럼 다수의 서로다른 환경설정이 필요한 프로젝트에 적합함.

 **Folder**

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

 **Multibranch Pipeline**

Creates a set of Pipeline projects according to detected branches in one SCM repository.

 **Organization Folder**

Creates a set of multibranch project subfolders by scanning for repositories.

If you want to create a new item from other existing, you can use this option:

 Copy from

OK

3. 선택

Dashboard > example name > Configuration

Configure

General

Advanced Project Options

Pipeline

☐ Poll SCM ?

☐ Quiet period ?

☐ 빌드를 원격으로 유발 (예: 스크립트 사용) ?

Advanced Project Options

고급

Pipeline script

Script ?

1

try sample Pipeline...

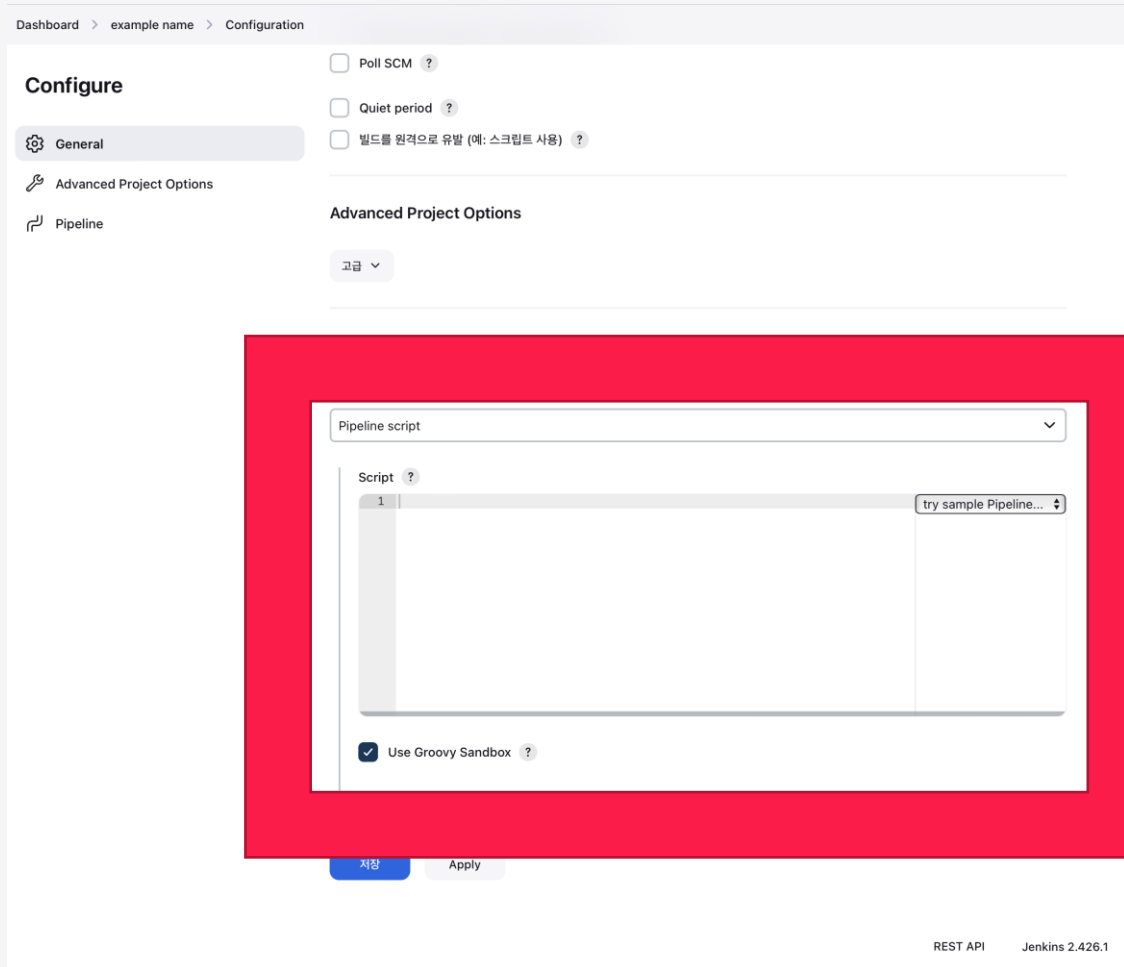
☒ Use Groovy Sandbox ?

저장

Apply

REST API

Jenkins 2.426.1



입력

```
node{  
  
    // init stage  
    stage("init"){  
        sh "echo init hello world"  
    }  
  
    // build stage  
    stage("build"){  
        sh "echo 'start build' "  
    }  
  
    // deploy stage  
    stage("deploy"){  
        sh "echo 'deploying' "  
    }  
}
```

Dashboard > test > Configuration

Configure

General

Advanced Project Options

Pipeline

☐ Poll SCM ?

☐ Quiet period ?

☐ 빌드를 원격으로 유발 (예: 스크립트 사용) ?

Advanced Project Options

고급 ▾

Pipeline

Definition

Pipeline script ▾

Script ?

1 node{

2

3 //init stage 시작

4 stage("init"){

5 sh "echo init"

6 }

7

8 //build stage 시작

9 stage("build"){

10 sh "echo 'start build' "

11 }

12

13 //deploy stage 시작

14 stage("deploy"){

15 sh "echo 'start deploy' "

16 }

☒ Use Groovy Sandbox ?

[Pipeline Syntax](#)

저장

Apply

REST API

Jenkins 2.426.1

Fast campus

Status

</>

Changes

▶

지금 빌드

⚙️

구성

🗑️

Pipeline 삭제

🔍

Full Stage View

✎

Rename

?

Pipeline Syntax

☀️

Build History

추이 ▼

No builds

📡

Atom feed (전체)

📡

Atom feed (일부)

example name

✎

상세 내용 입력

🔒

프로젝트 중지하기

Stage View

No data available. This Pipeline has not yet run.

고정링크

REST API

Jenkins 2.426.1

Status

</>

Changes

▶

지금 빌드

⚙️

구성

🗑️

Pipeline 삭제

🔍

Full Stage View

✎

Rename

?

Pipeline Syntax

☀️

Build History

추이 ▼

🔍

Filter builds...

/

🕒

2023. 12. 1. 오전 5:50

🔍

📡

Atom feed (전체)

📡

Atom feed (일부)

example name

✎

상세 내용 입력

🔒

프로젝트 중지하기

Stage View

Average stage times:

init	build
308ms	14ms

12월 01일

No Changes

14:50

308ms

14ms

고정링크

REST API

Jenkins 2.426.1

**Jenkins**

Q

검색 (%K)

?



1



jenkins

▼



로그아웃

Dashboard > example name > #1



Status



Changes



Console Output



View as plain text



Edit Build Information



Delete build '#1'



Replay



Pipeline Steps



Workspaces



콘솔 출력

Started by user [jenkins](#)

[Pipeline] Start of Pipeline

[Pipeline] node

Running on [Jenkins](#) in /var/jenkins_home/workspace/example name

[Pipeline] {

[Pipeline] stage

[Pipeline] { (init)

[Pipeline] sh

+ echo init hello world

init hello world

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (build)

[Pipeline] sh

+ echo start build

start build

[Pipeline] }

[Pipeline] // stage

[Pipeline] stage

[Pipeline] { (deploy)

[Pipeline] sh

+ echo deploying

deploying

[Pipeline] }

[Pipeline] // stage

[Pipeline] }

[Pipeline] // node

[Pipeline] End of Pipeline

Finished: SUCCESS

REST API

Jenkins 2.426.1

 Fast campus

3. Jenkins pipeline 예시

```
pipeline {
  agent any

  environment {
    DOCKER_IMAGE = 'your-docker-image-name'
  }

  stages {
    stage('Checkout') {
      steps {
        // Git 레포지토리에서 소스 코드 체크아웃
        git url: 'https://github.com/your-repository/your-project.git', branch: 'main'
      }
    }

    stage('Build Docker Image') {
      steps {
        script {
          // Docker 이미지 빌드
          docker.build("${DOCKER_IMAGE}:${env.BUILD_NUMBER}")
        }
      }
    }

    stage('Push Docker Image') {
      steps {
        script {
          // Docker Hub에 이미지 푸시
          docker.withRegistry('https://registry.hub.docker.com', 'dockerhub-credentials') {
            docker.image("${DOCKER_IMAGE}:${env.BUILD_NUMBER}").push()
          }
        }
      }
    }
  }
}
```